

Code and Architecture Example

1 Intro

ROS is an open-source robotics middleware suite. Although ROS is not an operating system (OS) but a set of software frameworks for robot software development, it provides services designed for a heterogeneous computer cluster such as hardware abstraction, low-level device control, implementation of commonly used functionality, message-passing between processes, and package management. (stole definition from Wikipedia)

2 Publisher Subscriber Architecture Model

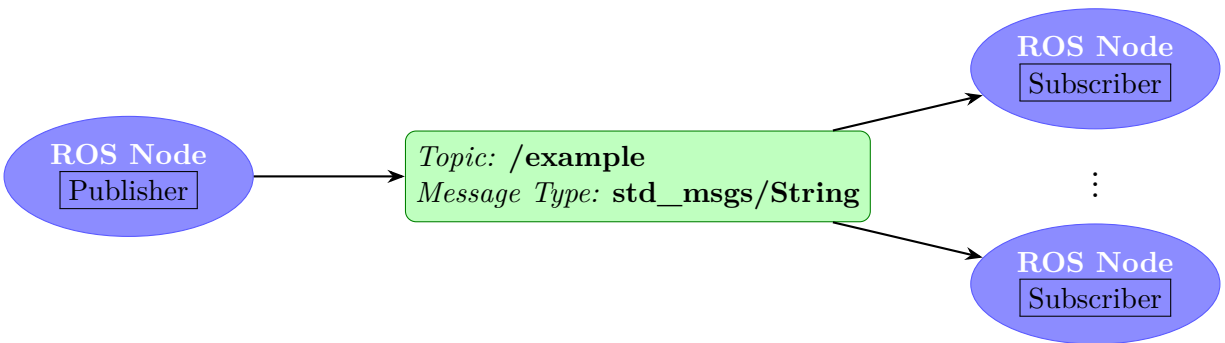


Figure 1: Publisher-subscriber architecture model.

Nodes: Are usually associated with a python script or c++ object. Nodes can do two main things - publish and subscribe. A node can be both a publisher and a subscriber.

Publisher node: Publish(send) data to a topic (represented by outgoing edges in figure above).

Subscriber node: Subscribes(listens) to data from a topic (represented by incoming edges in figure above), then uses a callback function to create desired output from topic data.

Topics: Are the name handlers for where nodes will publish and subscribe to data.

ROS messages: Are objects predefined in ROS or defined by the programmer that can be transmitted over topics.

3 System Design

ROS programming is often done using a top to bottom approach. The engineer must design the system architecture (as shown above), then develop (code) the individual nodes of the system.

Advantages:

1. Simplifies documentation and readability of the system.
2. Stores code in modular nodes that can be easily added and removed to any system (increases code re-usability).
3. Increased programming efficiency since nodes can be written in different languages and are allowed to communicate within a ROS environment.
4. ROS nodes communicate using asynchronous message passing, which improves performance by simplifying parallelism and making it more accessible to developers.

4 Whats a callback function?

Publisher nodes send data to topics, and subscriber nodes collect data from topics. Each time a subscriber node receives data from a topic, the subscriber node will run a callback function. A callback function is a function that receives data (input) from a topic asynchronously, and computes the output in a separate thread. Each time data is received from a topic a new thread opens and runs the callback function.

5 Whats a point cloud?

A point cloud is a set of 3D points within some frame. Example devices that gather point cloud data are Kinect (Xbox), lidar (similar to radar), and stereo camera (uses two RGB cameras to triangulate 3D points).

6 Example of architecture

Design a system architecture using the publisher-subscriber model that, from 2D object inferences, moves the robot's head in the direction of individual people. If there is more than one person, the robot creates a list of centroids (representing the people) and iterates through this list by pointing its head at each person for three seconds. The robot has two measuring devices: a camera which outputs RGB images of what the robot is looking at, and a Kinect device that outputs a point cloud of 3D points measured in the same frame as the camera.

7 Relating your first run to the publisher subscriber model

Remember to run in each terminal you open:

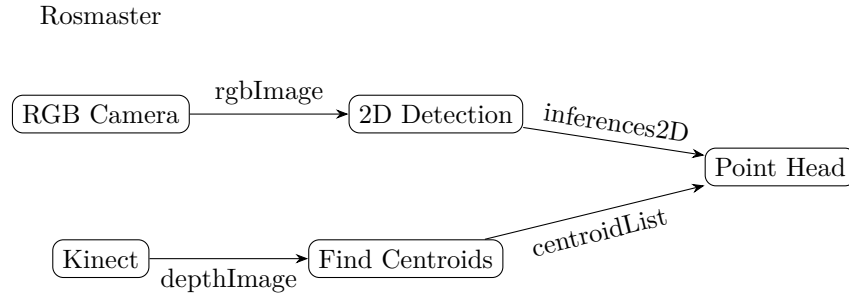


Figure 2: Example ROS architecture.

1. `cd catkin_ws`
2. `source devel/setup.bash`

Roscore- Running `roscore` in a separate terminal will set up your ROS master node for the system. The ROS master node is a node that manages all other nodes and topics created by the user.

roswarn- This command runs and adds a node to your architecture.

Examples (run each in a new terminal):

```
roslaunch talk_listen listener.py
roslaunch talk_listen talker.py
```

The argument `talk_listen` is a package; for now think of a package as a set of nodes that defines an architecture.

`listener.py` is the python script that runs the node listener, which subscribes to topic `chatter`.

`talker.py` is the python script that runs the node talker, which publishes to the topic `chatter`. (code provided on blackboard please review!)

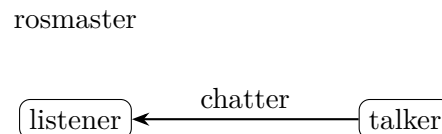


Figure 3: Listener/talker architecture.

To view this architecture in ROS (for debugging and making sure everything is correct) run the command:

```
roslaunch rqt_graph rqt_graph
```

8 Turtlesim

First lets start by running the `roslaunch` command. The `roslaunch` command allows you to launch multiple nodes and the rosmaster with just one command.

Example:

```
roslaunch turtle_motion turtle_clean.launch
```

(remember to create the package as instructed in the ROS Instructions PDF available on black-board.)

If nothing happens on the turtle simulator, open up the file `catkin_ws/src/turtle_motion/src/grid_clean.py` and call any of the functions included from there in the main function.

This command will run your rosmaster, `roslaunch turtlesim turtlesim_node`, and `roslaunch turtle_motion grid_clean.py` all in one terminal and you do not have to worry about managing multiple nodes across multiple terminals.

The important things to note about this file is that you must start and end the file with the launch tag (`<launch>`). Each node you set up begins with a node tag (`<node>`) and then you must specify the package where the node is contained (`pkg=`), the name of the file that sets up the node (`type=`), and the name of the node (`name=`); all other arguments are optional.

The launch-file image in the original document is rewritten below as editable LaTeX code:

```
1 <launch>
2   <node pkg="turtlesim" type="turtlesim_node"
3     name="turtlesim_node" output="screen"/>
4   <node pkg="turtle_motion" type="grid_clean.py"
5     name="grid_cleaner_node" output="screen"/>
6 </launch>
```

The original document next shows how the node, publisher and subscriber are set up. Every time you set up a publisher or subscriber you must provide both a topic name and a message type. Example: `rospy.Subscriber(topic_name, msg_type, callback_function_name)`. The callback function is written by the programmer, and it is the code executed every time the node receives a message on a topic.

```
1 if __name__ == '__main__':
2     try:
3         rospy.init_node("grid_clean", anonymous=True)
4         vel_pub = rospy.Publisher('/turtle1/cmd_vel', Twist,
5             queue_size=10)
6         pose_sub = rospy.Subscriber('/turtle1/pose', Pose,
7             poseCallback)
8         time.sleep(2)
9
10    except rospy.ROSInterruptException:
11        rospy.loginfo("node terminated")
```

Example of callback in code that updates variables `x`, `y`, and `yaw` based on the robot's current position on the simulator:

```
1 def poseCallback(pose_message):
2     global x, y, yaw
3     x = pose_message.x
4     y = pose_message.y
5     yaw = pose_message.theta
```

The move function has the robot move at some speed `linear.x` by publishing to the `/turtle1/cmd_vel` topic. The message type of this topic is `geometry_msgs/Twist`. This message has two member variables, `Vector3 linear` and `Vector3 angular`. The linear variable sets the linear velocity of the turtle and the angular variable sets the angular velocity of the turtle robot. In the code we use the distance formula to compute how much the robot has moved from its initial position.

```
1 def move(speed, distance, is_forward):
2     vel_msg = Twist()
3     x0 = x
4     y0 = y
5
6     if is_forward:
7         vel_msg.linear.x = abs(speed)
8     else:
9         vel_msg.linear.x = -abs(speed)
10
11     distance_moved = 0.0
12     loop_rate = rospy.Rate(10)
13
14     while distance_moved < distance:
15         vel_pub.publish(vel_msg)
16         loop_rate.sleep()
17         distance_moved = abs(math.sqrt(((x-x0)**2) + ((y-y0)**2)))
```

The rotate function also publishes to the `/turtle1/cmd_vel` topic and uses a message type of `geometry_msgs/Twist`. This time we are interested in the member variable `Vector3 angular`, which sets the rotational velocity of the robot. The speed input parameter tells us how fast our turtle robot will rotate per second, and the idea is to rotate for however many seconds are required to reach our goal.

```
1 def rotate(angular_speed_degree, relative_angle_degree, clockwise):
2     vel_msg = Twist()
3     angular_speed = math.radians(abs(angular_speed_degree))
4
5     if clockwise:
6         vel_msg.angular.z = -abs(angular_speed)
7     else:
8         vel_msg.angular.z = abs(angular_speed)
9
10    loop_rate = rospy.Rate(10)
11    current_angle_degree = 0.0
12    t0 = rospy.Time.now().to_sec()
13
14    while current_angle_degree < relative_angle_degree:
15        vel_pub.publish(vel_msg)
16        loop_rate.sleep()
17        t1 = rospy.Time.now().to_sec()
18        current_angle_degree = (t1-t0) * angular_speed_degree
```

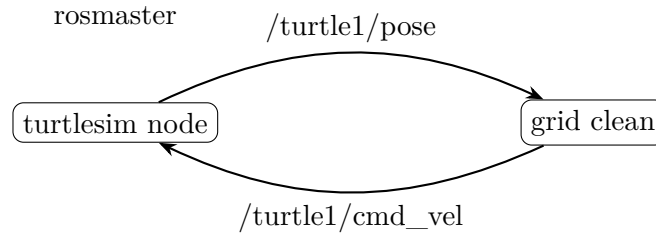


Figure 4: Turtlesim architecture.

8.1 turtle architecture

9 Website Server ROS Setup

1. Create an account on theconstructsim.com.
2. In the left hand column, click on my “rosjects” then click on create “new rosject”.
3. Name your rosject whatever you would like but make sure to select ROS distro as “ROS melodic”.
4. There will be a tool bar at the bottom of the screen when you open your project. From this tool bar you will be using the following: Web shell, Code editor, graphical tools.

10 Environment Setup

1. Create a folder named `catkin_ws`. Create a sub-folder inside of `catkin_ws` named `src`. `catkin_ws` will be your local working environment for ROS.
2. Run `cd ~/catkin_ws` followed by `catkin_make`. The command `catkin_make` converts a regular folder into a ROS repository.
3. Every time you open a web shell or new tabs in web shell run `source ~/catkin_ws/devel/setup.bash`. This command sets your working environment in the current shell to the repo `catkin_ws`. Otherwise you will be running from its installation working environment.

11 Creating ROS packages

1. A ROS package is a small standalone ROS project created inside of your ROS repository.
2. Run `cd ~/catkin_ws/src` followed by `catkin_create_pkg [name of package] [depend1] [depend2] ...`

An example (Do it!):

```
catkin_create_pkg talk_listen std_msgs roscpp rospy
```

3. After creating a package make sure to execute steps 2 and 3 from section Environment Setup.

12 First Example

1. On blackboard under course materials you will find a folder called `ros_examples`, and inside this folder another folder called `talk_listen_stuff`. Copy the files `listener.py` and `talker.py` into `~/catkin_ws/src/talk_listen/src`.
2. Make sure files are executable by running:

```
chmod +x ~/catkin_ws/src/talk_listen/src/listener.py
chmod +x ~/catkin_ws/src/talk_listen/src/talker.py
```

3. In your current shell run `roscore`; this will set up a ROS master node that will manage all future nodes.
4. Open a new tab in your web shell, run `source ~/catkin_ws/devel/setup.bash` followed by `roslaunch talk_listen listener.py`.
5. Open a third tab in your web shell, run `source ~/catkin_ws/devel/setup.bash` followed by `roslaunch talk_listen talker.py`.
6. The steps in this section should result in basic stdout output in the terminal where you ran the listener node (`roslaunch talk_listen listener.py`). Congrats, you just ran your first ROS system!

13 More

Moving forward you will just need to create two more packages for the homework (run these in `~/catkin_ws/src`):

1. `catkin_create_pkg turtle_motion std_msgs roscpp rospy`
2. `catkin_create_pkg learn_tf2 tf2 tf2_ros roscpp rospy turtlesim`

Use the examples in `ros_examples` (on blackboard) to test these projects out.